

Index

Fast I/O (put at top of main)	3	Basic Constructive Algorithms	10
Factorial / Inverse Factorial	3	Digit DP	11
A — Problem Solving Basics	3	Permutations and Permutation Cycles . . .	11
Invariants and Monovariants	3	The Slime Trick (optional stretch)	11
Backtracking	3	A — Linear Recurrences	12
A — Number Theory	3	Matrix Exponentiation	12
Binary Exponentiation & Modular Arithmetic	3	Fibonacci Numbers	12
Fermat's Little Theorem & Modular Inverse	4	Linear Recurrence Relations	12
Euclidean Algorithm	4	Linear Recurrence via Cayley-Hamilton	
Sieve, Prime Factorization and Divisors	4	$O(k^2 \log n)$	12
Extended Euclid	4	A — Geometry (2D)	13
Bezout's Identity	5	Points, Vectors, Dot/Cross, Polar Sort . .	13
Chinese Remainder Theorem (CRT)	5	Lines, Segments, Distance, Projections,	
Euler's Totient Function	5	Reflections	13
Lucas Theorem	5	Polygons	13
A — Combinatorics	6	Convex Hull (2D)	14
Permutations, Combinations & Basic Counting	6	A — Polynomial / FFT	14
Stars and Bars	6	Fast Fourier Transform (FFT)	14
Pigeonhole Principle	6	Number Theoretic Transform (NTT)	15
Contribution Technique	6	Generating Functions	16
Principle of Inclusion and Exclusion (PIE)	7	B — Array Techniques	16
Derangements	7	Difference Array	16
Inclusion Exclusion on Multiples	7	Sliding Window Technique	16
Stirling Numbers (Basic)	7	Two Pointers	17
A — Probability & Expected Value	8	Coordinate Compression	17
Probabilities and Expected Values	8	Direction Array	17
Expected Value Powers Technique	8	B — Graph Basics	17
Gambler's Ruin	8	Depth First Search (DFS)	17
A — Greedy	8	Breadth First Search (BFS)	18
Basic Greedy Algorithms	9	Topological Sorting	18
Exchange Arguments	9	Tree Diameter	18
Activity Selection Problem	9	Cycle Detection	18
Fractional Knapsack Problem	9	DSU / Union Find	19
A — Game Theory	10	Minimum Spanning Tree (Prim's and	
Grundy Number and Basic Game Theory	10	Kruskal's)	19
Nimber (optional/advanced)	10	B — Searching & Sorting	20
Games on Arbitrary Graphs (optional/advanced)	10	Binary Search (incl. binary-search-on-answer)	20
A — DP-ish / Misc	10	Ternary Search / Binary Search on Answer	20
		C++ STL Sort and Custom Comparator	20
		B — Dynamic Programming	20

Knapsack and Basic Dynamic Programming	20	Aho Corasick	27
Longest Increasing Subsequence (LIS)	21	B — Flows & Matching	27
Interval DP / Range DP / Matrix Chain Multiplication	21	Maximum Flow / Minimum Cut (Dinic's)	27
Edit Distance	21	Bipartite Matching (Hopcroft-Karp and Kuhn's)	28
Bitmask DP	21	B — Advanced Graph	29
DP on Trees and DAGs	22	Strongly Connected Components (SCC, Kosaraju)	29
Sum Over Subsets DP (SOS DP)	22	2-SAT	29
B — Data Structures	22	Articulation Bridges and Bridge Tree	29
Segment Tree (Point Update Range Query)	22	Articulation Points and Block Cut Tree	30
Segment Tree with Lazy Propagation	23	Heavy Light Decomposition (HLD) —	
BIT / Fenwick Tree	23	Subtrees and Path Query	30
Lower bound on BIT (BIT binary search)	24	Centroid Decomposition	30
Sparse Table	24	Binary Lifting and LCA	31
Basic SQRT Decomposition	24	LCA in $O(1)$ (optional stretch, sparse table on Euler tour)	31
Splitting Objects into Light and Heavy / Case Processing	25	B — DP Optimization	32
Trie on Strings and Bits	25	Convex Hull Trick (CHT) and Dynamic CHT	32
B — Strings	26	Knuth Optimization	32
String Hashing	26	Divide and Conquer Optimization	32
Knuth Morris Pratt (KMP) and Prefix Automaton	26		
Z algorithm	26		

Fast I/O (put at top of main)

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
cout.tie(NULL);
```

Factorial / Inverse Factorial

```
vector<ll> fact(MAXN+1,1), invfact(MAXN+1,1);
for(ll i=1;i<=MAXN;i++) fact[i]=fact[i-1]*i%mod;
invfact[MAXN]=modpow(fact[MAXN],mod-2);
for(ll i=MAXN;i>=1;i--) invfact[i-1]=invfact[i]*i%mod;
```

A — Problem Solving Basics

Invariants and Monovariants

```
// reasoning technique, not a template.
// find quantity Q(state) that is fixed (invariant)
// or strictly monotonic (monovariant) under each move;
// e.g. parity of #coins, sum mod k, max-element, etc.
// ex: Q = (sum of array) % 2 is invariant under swaps.
```

No fixed I/O — used to prove a state is unreachable, or bound the number of operations till termination.

Backtracking

```
bool ok[MAXN];
void bt(ll pos, vector<ll>&cur){
    if(pos==n){ /* process cur */ return; }
    for(ll v=1;v<=n;v++){
        if(ok[v]) continue;
        ok[v]=1; cur.push_back(v);
        bt(pos+1,cur);
        cur.pop_back(); ok[v]=0;
    }
}
```

I/O: n and problem-specific constraints → enumerate/count/find a feasible configuration. Prune inside the loop as early as possible.

A — Number Theory

Binary Exponentiation & Modular Arithmetic

```
ll modpow(ll a, ll b, ll m=mod){
    a%=m; if(a<0) a+=m;
    ll r=1;
```

```

while(b>0){
    if(b&1) r=r*a%m;
    a=a*a%m; b>>=1;
}
return r;
}

```

I/O: $a, b, m \rightarrow a^b \bmod m$ in $O(\log b)$.

Fermat's Little Theorem & Modular Inverse

```

ll inv(ll a, ll m=mod){ return modpow(a,m-2,m); } // m prime
// a/b mod m == a * inv(b) mod m

```

I/O: a (and prime modulus m) $\rightarrow a^{-1} \bmod m$.

Euclidean Algorithm

```

ll gcd(ll a, ll b){ return b? gcd(b,a%b): a; }
ll lcm(ll a, ll b){ return a/gcd(a,b)*b; }

```

I/O: $a, b \rightarrow \gcd(a, b) / \text{lcm}(a, b)$.

Sieve, Prime Factorization and Divisors

```

bool comp[MAXN]; vector<ll> primes, spf(MAXN);
void sieve(){
    for(ll i=2;i<MAXN;i++){
        if(!comp[i]){ primes.push_back(i); spf[i]=i; }
        for(ll p:primes){
            if(i*p>=MAXN) break;
            comp[i*p]=1; spf[i*p]=p;
            if(i%p==0) break;
        }
    }
}
vector<pair<ll,ll>> factorize(ll x){ // using spf, x<MAXN
    vector<pair<ll,ll>> f;
    while(x>1){ ll p=spf[x],c=0; while(x%p==0){x/=p;c++;} f.push_back({p,c}); }
    return f;
}

```

Input: build sieve once up to MAXN. factorize(x) returns prime $\rightarrow$ exponent pairs;

#divisors = $\prod (c_i + 1)$, sum of divisors = $\prod \frac{p_i^{c_i+1} - 1}{p_i - 1}$.

Extended Euclid

```

ll extgcd(ll a, ll b, ll &x, ll &y){
    if(!b){ x=1; y=0; return a; }
    ll x1,y1; ll g=extgcd(b,a%b,x1,y1);
    x=y1; y=x1-(a/b)*y1;
}

```

```
    return g;
}
```

Input: a, b . Finds x, y with $ax + by = \gcd(a, b)$ by back-substituting the Euclidean steps. Output: (g, x, y) via return value and references.

Bezout's Identity

```
// extgcd(a,b,x,y) gives one solution (x,y) to  $ax+by=g=\gcd(a,b)$ .
// all solutions:  $x+k*(b/g)$ ,  $y-k*(a/g)$  for integer  $k$ .
//  $a*x+b*y=c$  solvable iff  $g|c$ ; scale  $(x,y)$  by  $c/g$ .
```

Input: a, b, c . Output: whether $ax + by = c$ has an integer solution ($\gcd(a, b) \mid c$), and the general solution family.

Chinese Remainder Theorem (CRT)

```
// merge  $x = r_1 \pmod{m_1}$ ,  $x = r_2 \pmod{m_2} \rightarrow x = r \pmod{\text{lcm}(m_1, m_2)}$ 
pair<ll, ll> crt(ll r1, ll m1, ll r2, ll m2){
    ll x, y; ll g = extgcd(m1, m2, x, y);
    if((r2 - r1) % g) return {0, -1}; // no solution
    ll lcm = m1 / g * m2;
    ll t = ((r2 - r1) / g * x) % (m2 / g);
    ll r = ((r1 + m1 * t) % lcm + lcm) % lcm;
    return {r, lcm};
}
```

Input: pairs $(r_1, m_1), (r_2, m_2)$ of a congruence system (fold pairwise over more equations). Output: (r, lcm) with $x \equiv r \pmod{\text{lcm}}$, or $\text{lcm} = -1$ if infeasible.

Euler's Totient Function

```
ll phi(ll n){
    ll r = n;
    for(ll p = 2; p * p <= n; p++){
        if(n % p == 0){
            while(n % p == 0) n /= p;
            r -= r / p;
        }
    }
    if(n > 1) r -= r / n;
    return r;
}
```

I/O: $n \rightarrow \phi(n)$ = count of integers in $[1, n]$ coprime to n , via trial division of n 's prime factors.

Lucas Theorem

```
ll C(ll n, ll r, ll p){ //  $n, r < p$ ,  $p$  prime, small
    if(r > n) return 0;
    return fact[n] * invfact[r] % p * invfact[n - r] % p; // fact built mod p
}
```

```

11 lucas(11 n, 11 r, 11 p){
    if(r==0) return 1;
    return C(n%p,r%p,p)*lucas(n/p,r/p,p)%p;
}

```

Input: n, r and small prime p . Computes $\binom{n}{r} \bmod p$ by splitting n, r into base- p digits. Output: $\binom{n}{r} \bmod p$.

A — Combinatorics

Permutations, Combinations & Basic Counting

```

11 nPr(11 n, 11 r){ if(r>n||r<0) return 0; return fact[n]*invfact[n-r]%mod; }
11 nCr(11 n, 11 r){ if(r>n||r<0) return 0; return fact[n]*invfact[r]%mod*invfact[n-r]%mod; }

```

I/O: n, r (needs fact/invfact precomputed) $\rightarrow {}^n P_r$ or $\binom{n}{r} \bmod m$.

Stars and Bars

```

// #ways to write n as ordered sum of k non-negative ints = C(n+k-1,k-1)
// with each part >=1: C(n-1,k-1)
11 starsAndBars(11 n, 11 k){ return nCr(n+k-1,k-1); }

```

I/O: n (total), k (parts) $\rightarrow$ #ways to split n identical items into k (possibly empty) groups.

Pigeonhole Principle

```

// n items into m boxes,  $n > m*k \Rightarrow$  some box has  $> k$  items.
// used as an existence argument, no code to run.

```

No fixed I/O — argue existence of a repeated/clustered value from counting bounds (e.g. prefix sums mod m repeat within $m+1$ terms).

Contribution Technique

```

// swap order of summation: instead of summing f(subset)/f(pair) directly,
// for each element/pair compute how many times & with what sign it is
// counted across all subsets/pairs, sum contributions instead.
// e.g. sum of subarray sums = sum_i a[i]*(i+1)*(n-i)
11 sumOfSubarraySums(vector<11>&a){
    11 n=a.size(), res=0;
    for(11 i=0;i<n;i++) res+=a[i]*(i+1)*(n-i);
    return res;
}

```

I/O: array $a[0..n-1] \rightarrow$ sum over all subarrays of (subarray sum), by counting how many subarrays each $a[i]$ lies in.

Principle of Inclusion and Exclusion (PIE)

```
// |A1 ∪ ... ∪ Ak| = sum|Ai| - sum|Ai^Aj| + sum|Ai^Aj^Ak| - ...
ll pieOverMasks(vector<ll>&cnt /*cnt[mask]=|intersection of sets in mask|*/, ll k){
    ll res=0;
    for(ll mask=1; mask<(1LL<<k); mask++){
        ll bits=__builtin_popcountll(mask);
        res += (bits&1? 1: -1) * cnt[mask];
    }
    return res;
}
```

Input: k sets and, for each nonempty mask, size of the intersection of the chosen sets. Computes alternating sum over all 2^k-1 masks. Output: $|A_1 \cup \dots \cup A_k|$.

Derangements

```
ll der[MAXN];
void buildDerangements(){
    der[0]=1; der[1]=0;
    for(ll i=2; i<MAXN; i++) der[i]=(i-1)*(der[i-1]+der[i-2])%mod;
}
```

I/O: $n \rightarrow D_n = \#$ permutations of n with no fixed point, via $D_n = (n-1)(D_{n-1} + D_{n-2})$.

Inclusion Exclusion on Multiples

```
// count numbers in [1,n] divisible by none of p[0..k-1] (distinct primes/divisors)
ll countCoprimeToAll(ll n, vector<ll>&p){
    ll k=p.size(), res=0;
    for(ll mask=0; mask<(1LL<<k); mask++){
        ll prod=1, bits=0;
        for(ll i=0; i<k; i++) if(mask&(1LL<<i)){ prod*=p[i]; bits++; }
        res += (bits&1? -1:1) * (n/prod);
    }
    return res;
}
```

Input: n and a list of (coprime) divisors. Sums $n/\prod(\text{chosen divisors})$ with inclusion-exclusion signs over all subsets. Output: count of $1..n$ divisible by none.

Stirling Numbers (Basic)

```
ll stirling2[MAXS][MAXS]; // S(n,k): partitions of n items
                          // into k non-empty unlabeled groups
void buildStirling2(){
```

```

stirling2[0][0]=1;
for(ll n=1;n<MAXS;n++)
    for(ll k=1;k<=n;k++)
        stirling2[n][k]=(k*stirling2[n-1][k]+stirling2[n-1][k-1])%mod;
}

```

I/O: $n, k \rightarrow S(n, k)$ (2nd kind) via $S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1)$.

A — Probability & Expected Value

Probabilities and Expected Values

*// linearity of expectation: $E[X_1 + \dots + X_n] = E[X_1] + \dots + E[X_n]$, even if dependent.
 // often: define indicator $X_i = [\text{event } i \text{ happens}]$, sum $E[X_i] = P(\text{event } i)$.
 double ans=0;
 for(ll i=0;i<n;i++) ans += prob[i]; // $E[\text{\#events}] = \text{sum of individual probs}$*

I/O: per-item/event probabilities $\rightarrow$ overall expectation, by summing individual (indicator) expectations instead of reasoning about the joint distribution.

Expected Value Powers Technique

*// $E[X] = \sum_{t=1}^{\infty} P(X \geq t)$ (for non-negative integer X)
 // useful when $P(X \geq t)$ is easy but the pmf $P(X=t)$ is not.
 double E=0;
 for(ll t=1;t<=maxT;t++) E += PgeT(t);*

Input: a way to compute $P(X \geq t)$ for a non-negative integer r.v. X . Output: $E[X] = \sum_{t \geq 1} P(X \geq t)$.

Gambler's Ruin

*// random walk on $0..N$, $p = P(\text{step } +1)$, $q=1-p$, absorbing at 0 and N .
 // $P(\text{reach } N \text{ before } 0 \mid \text{start } i)$, $p \neq q$:
 double ruinProb(ll i, ll N, double p){
 double q=1-p, r=q/p;
 return (1-pow(r,i))/(1-pow(r,N));
 }*

Input: start i , boundary N , step probability p . Output: probability of reaching N before 0 (use i/N directly if $p = q = 0.5$).

A — Greedy

Basic Greedy Algorithms

```
// pick the locally-best option at each step; correctness needs proof
// (exchange argument / matroid / problem-specific).
sort(a.begin(),a.end(), /*comparator by the greedy key*/ greater<ll>());
for(auto &x: a){ /* decide greedily, update state */ }
```

I/O: items with some value/cost $\rightarrow$ sort by the right key, then sweep once making the locally optimal choice.

Exchange Arguments

```
// to prove a greedy order is optimal: take any optimal solution,
// show swapping two adjacent out-of-order items doesn't worsen it.
// => some greedy order is also optimal, so sorting by that key works.
```

No fixed I/O — a proof technique: pairwise adjacent swap shows the greedy ordering is at least as good as any other.

Activity Selection Problem

```
bool cmp(pair<ll,ll>&a, pair<ll,ll>&b){ return a.second<b.second; }
ll activitySelection(vector<pair<ll,ll>> iv){ // {start,end}
    sort(iv.begin(),iv.end(),cmp);
    ll cnt=0, last=-1e18;
    for(auto&[s,e]:iv) if(s>=last){ cnt++; last=e; }
    return cnt;
}
```

I/O: array of (start,end) intervals $\rightarrow$ max #non-overlapping intervals, by sorting on end time and sweeping.

Fractional Knapsack Problem

```
double fracKnapsack(ll W, vector<pair<ll,ll>>&items){ // {value,weight}
    sort(items.begin(),items.end(),[](auto&a,auto&b){
        return a.first*(double)b.second > b.first*(double)a.second; // value/weight desc
    });
    double res=0;
    for(auto&[v,w]:items){
        if(W>=w){ res+=v; W-=w; }
        else { res += v*(double)W/w; break; }
    }
    return res;
}
```

I/O: capacity W , items (value,weight) $\rightarrow$ max value taking fractional amounts, by sorting on value/weight ratio.

A — Game Theory

Grundy Number and Basic Game Theory

```

11 grundy[MAXN]; bool vis[MAXN];
11 G(11 state){
    if(vis[state]) return grundy[state];
    vis[state]=1;
    set<11> s;
    for(auto nxt: movesFrom(state)) s.insert(G(nxt));
    11 g=0; while(s.count(g)) g++;
    return grundy[state]=g;
}

```

// combined game (sum of independent games) is a win iff XOR of grundy != 0

I/O: a game state (and its move function) $\rightarrow$ Grundy number (mex of reachable states' Grundy numbers); XOR across independent sub-games to decide first-player win.

Nimber (optional/advanced)

*// nimber (mex-based) addition IS xor; nimber multiplication is NOT *,
 // needs a custom table built via the mex of a recursive recurrence
 // (rarely needed in full generality for ICPC -- usually plain grundy/XOR suffices).*

Used when games combine via nim-multiplication, not just nim-addition (XOR); normally only the grundy+XOR rule above is needed.

Games on Arbitrary Graphs (optional/advanced)

```

// token on a DAG node, players move along edges, last to move wins:
// a node is LOSING (P-position) iff every outgoing edge leads to a WINNING node.
bool win[MAXN]; // computed on reverse-topological / via DFS memo
bool W(11 u){
    if(vis[u]) return win[u]; vis[u]=1;
    for(11 v: g[u]) if(!W(v)) return win[u]=true;
    return win[u]=false;
}

```

Input: DAG (or finite graph with no draws) and start node. Computes win/lose by DFS memo (a node wins if some neighbour loses). Output: true if the player to move from start wins.

A — DP-ish / Misc

Basic Constructive Algorithms

*// build an explicit valid configuration instead of searching/counting.
 // common patterns: handle small/edge cases separately; build greedily
 // from one end; find a repeating/periodic block and tile it;
 // work backwards from the target state.*

No fixed I/O — construct any one valid answer directly (often via a clean pattern), then verify it satisfies every given constraint.

Digit DP

```
ll dp[20][2][STATE]; string s;
ll go(ll pos, bool tight, ll state){
    if(pos==(ll)s.size()) return baseCase(state);
    if(!tight && dp[pos][tight][state]!=-1) return dp[pos][tight][state];
    ll lim = tight? s[pos]-'0': 9, res=0;
    for(ll d=0; d<=lim; d++){
        res += go(pos+1, tight&&(d==lim), nextState(state,d));
    }
    if(!tight) dp[pos][tight][state]=res;
    return res;
}
```

I/O: upper bound as digit string $s \rightarrow$ count of numbers in $[0, s]$ satisfying a digit-wise property, via memoized DFS over (position, tight, state).

Permutations and Permutation Cycles

```
vector<ll> cycleLens(vector<ll>&p){ // p[i] = where i maps to, 0-indexed
    ll n=p.size(); vector<bool> vis(n); vector<ll> lens;
    for(ll i=0;i<n;i++){ if(!vis[i]){
        ll len=0, j=i;
        while(!vis[j]){ vis[j]=1; j=p[j]; len++; }
        lens.push_back(len);
    }
    return lens;
}
```

I/O: permutation $p[0..n-1] \rightarrow$ lengths of each cycle (sum = n); min swaps to sort = $n - \#cycles$.

The Slime Trick (optional stretch)

*// merging-cost DP (e.g. merge n piles, cost=sum of merged sizes),
 // when any subset order is merge-able: cost is independent of order
 // if cost(a,b)=a+b, so total = sum over each element of (depth in merge tree)*value.
 // usually solved via DP on intervals (see Interval DP) or a priority queue
 // (always merge two smallest), matching Huffman-coding structure.*

No single fixed I/O — typically reduces to interval DP (contiguous slimes) or a min-heap greedy merge (unordered), minimizing total merge cost.

A — Linear Recurrences

Matrix Exponentiation

```
typedef vector<vector<ll>> mat;
mat mul(mat&A, mat&B){
    ll n=A.size(), m=B[0].size(), k=B.size();
    mat C(n, vector<ll>(m,0));
    for(ll i=0;i<n;i++) for(ll j=0;j<k;j++) if(A[i][j])
        for(ll l=0;l<m;l++) C[i][l]=(C[i][l]+A[i][j]*B[j][l])%mod;
    return C;
}
mat matpow(mat A, ll p){
    ll n=A.size(); mat R(n, vector<ll>(n,0));
    for(ll i=0;i<n;i++) R[i][i]=1;
    while(p>0){ if(p&1) R=mul(R,A); A=mul(A,A); p>>=1; }
    return R;
}
```

I/O: square transition matrix A and power $p \rightarrow A^p \bmod m$, computed by repeated squaring, $O(k^3 \log p)$.

Fibonacci Numbers

```
ll fib(ll n){ // via matrix power of [[1,1],[1,0]]
    mat A={{1,1},{1,0}};
    mat R=matpow(A,n);
    return R[0][1]; // F(n)
}
```

I/O: $n \rightarrow F(n) \bmod m$ in $O(\log n)$ via $[[1,1],[1,0]]^n$.

Linear Recurrence Relations

```
// x_n = c1*x_{n-1} + c2*x_{n-2} + ... + ck*x_{n-k}
// build companion matrix (k x k) and matrix-power it (see above),
// or compute with Kitamasa / Cayley-Hamilton for very large n, small k.
mat companion(vector<ll>&c){ // c[0..k-1] = coefficients
    ll k=c.size(); mat M(k, vector<ll>(k,0));
    for(ll j=0;j<k;j++) M[0][j]=c[j];
    for(ll i=1;i<k;i++) M[i][i-1]=1;
    return M;
}
```

Input: coefficients $c_1..c_k$ and first k terms, target index n . Builds the $k \times k$ companion matrix and raises it to the $(n-k+1)$ -th power. Output: $x_n \bmod m$.

Linear Recurrence via Cayley-Hamilton $O(k^2 \log n)$

```
// represent x_n as a poly in the recurrence's char. poly mod x^k;
// compute x^n mod charPoly(x) via poly multiplication ( $O(k^2)$  each),
```

*// then $x_n = \sum \text{coeff}[i] * x_i$ ($i < k$, known initial terms).
 // use when k up to ~ 2000 - 5000 and n up to $1e18$ (matrix power's k^3 too slow).*

*Input: recurrence order k , coefficients, initial terms, target n . Reduces $x^n \bmod m$ to the characteristic polynomial via repeated-squaring polynomial multiplication.
 Output: $x_n \bmod m$ in $O(k^2 \log n)$ instead of $O(k^3 \log n)$.*

A — Geometry (2D)

Points, Vectors, Dot/Cross, Polar Sort

```
typedef pair<double,double> pt;
#define X first
#define Y second
pt operator-(pt a, pt b){ return {a.X-b.X, a.Y-b.Y}; }
double dot(pt a, pt b){ return a.X*b.X + a.Y*b.Y; }
double cross(pt O, pt A, pt B){ return (A.X-O.X)*(B.Y-O.Y)-(A.Y-O.Y)*(B.X-O.X); }
void polarSort(vector<pt>&p, pt origin){
    sort(p.begin(),p.end(),[&](pt a, pt b){
        return atan2(a.Y-origin.Y,a.X-origin.X) < atan2(b.Y-origin.Y,b.X-origin.X);
    });
}
```

I/O: points $(x,y) \rightarrow$ dot/cross products (sign of cross tells turn direction / orientation), or points sorted by angle around an origin.

Lines, Segments, Distance, Projections, Reflections

```
double distPtSeg(pt p, pt a, pt b){
    pt ab={b.X-a.X,b.Y-a.Y}, ap={p.X-a.X,p.Y-a.Y};
    double t = dot(ap,ab) / dot(ab,ab);
    t = max(0.0, min(1.0, t));
    pt proj = {a.X+t*(b.X-a.X), a.Y+t*(b.Y-a.Y)};
    return hypot(p.X-proj.X, p.Y-proj.Y);
}
bool segIntersect(pt a, pt b, pt c, pt d){
    double d1=cross(c,d,a), d2=cross(c,d,b), d3=cross(a,b,c), d4=cross(a,b,d);
    return ((d1>0)!=(d2>0)) && ((d3>0)!=(d4>0));
}
```

Input: segment endpoints (and a query point). Clamped projection gives point-to-segment distance; sign comparison of two cross products each gives proper segment intersection. Output: a distance / boolean.

Polygons

```
double polyArea(vector<pt>&p){ // signed area, CCW positive
    double a=0; ll n=p.size();
    for(ll i=0;i<n;i++) a += cross({0,0}, p[i], p[(i+1)%n]);
}
```

```

    return a/2;
}
bool pointInPoly(pt p, vector<pt>&poly){ // ray casting
    bool in=false; ll n=poly.size();
    for(ll i=0,j=n-1;i<n;j=i++){
        if(((poly[i].Y>p.Y)!=(poly[j].Y>p.Y)) &&
            (p.X < (poly[j].X-poly[i].X)*(p.Y-poly[i].Y)/(poly[j].Y-poly[i].Y)+poly[i].X))
            in=!in;
    }
    return in;
}

```

Input: polygon vertices in order (and a query point). Shoelace formula gives signed area (and CCW/CW); ray casting gives point-in-polygon. Output: area (double) / boolean.

Convex Hull (2D)

```

typedef pair<ll,ll> ipt;
ll crossI(ipt O, ipt A, ipt B){
    return (A.first-O.first)*(B.second-O.second)-(A.second-O.second)*(B.first-O.first);
}
vector<ipt> convexHull(vector<ipt> p){ // monotone chain, CCW, no dup/collinear
    boundary pts
    ll n=p.size(), k=0;
    if(n<3) return p;
    sort(p.begin(),p.end());
    vector<ipt> h(2*n);
    for(ll i=0;i<n;i++){
        while(k>=2 && crossI(h[k-2],h[k-1],p[i])<=0) k--;
        h[k++]=p[i];
    }
    for(ll i=n-2,t=k+1;i>=0;i--){
        while(k>=t && crossI(h[k-2],h[k-1],p[i])<=0) k--;
        h[k++]=p[i];
    }
    h.resize(k-1);
    return h;
}

```

Input: array of n integer points. Computes the convex hull via Andrew's monotone chain (sorts, builds lower+upper chain, drops collinear boundary points since ≤ 0). Output: hull vertices in CCW order.

A — Polynomial / FFT

Fast Fourier Transform (FFT)

```

typedef complex<double> cd;
void fft(vector<cd>&a, bool invert){
    ll n=a.size();

```

```

for(ll i=1,j=0;i<n;i++){
    ll bit=n>>1;
    for(;j&bit;bit>>=1) j^=bit;
    j^=bit;
    if(i<j) swap(a[i],a[j]);
}
for(ll len=2;len<=n;len<=<=1){
    double ang=2*M_PI/len*(invert?-1:1);
    cd w1(cos(ang),sin(ang));
    for(ll i=0;i<n;i+=len){
        cd w(1);
        for(ll j=0;j<len/2;j++){
            cd u=a[i+j], v=a[i+j+len/2]*w;
            a[i+j]=u+v; a[i+j+len/2]=u-v; w*=w1;
        }
    }
}
if(invert) for(cd&x:a) x/=n;
}
vector<ll> multiply(vector<ll>&a, vector<ll>&b){
    vector<cd> fa(a.begin(),a.end()), fb(b.begin(),b.end());
    ll n=1; while(n<(ll)(a.size()+b.size())) n<=<=1;
    fa.resize(n); fb.resize(n);
    fft(fa,false); fft(fb,false);
    for(ll i=0;i<n;i++) fa[i]*=fb[i];
    fft(fa,true);
    vector<ll> res(n);
    for(ll i=0;i<n;i++) res[i]=llround(fa[i].real());
    return res;
}

```

Input: two coefficient arrays a, b (polynomials). Pads to a power of two, transforms both to point-value form, multiplies pointwise, inverse-transforms back. Output: coefficient array of $a \cdot b$ (round to nearest integer).

Number Theoretic Transform (NTT)

```

const ll NTTMOD=998244353, G=3;
void ntt(vector<ll>&a, bool invert){
    ll n=a.size();
    for(ll i=1,j=0;i<n;i++){
        ll bit=n>>1;
        for(;j&bit;bit>>=1) j^=bit;
        j^=bit;
        if(i<j) swap(a[i],a[j]);
    }
    for(ll len=2;len<=n;len<=<=1){
        ll w=modpow(G, (NTTMOD-1)/len, NTTMOD);
        if(invert) w=modpow(w,NTTMOD-2,NTTMOD);
        for(ll i=0;i<n;i+=len){
            ll wn=1;
            for(ll j=0;j<len/2;j++){
                ll u=a[i+j], v=a[i+j+len/2]*wn%NTTMOD;
                a[i+j]=(u+v)%NTTMOD; a[i+j+len/2]=(u-v+NTTMOD)%NTTMOD;
                wn=wn*w%NTTMOD;
            }
        }
    }
}

```

```

    }
  }
  if(invert){ ll ninv=modpow(n,NTTMOD-2,NTTMOD); for(ll&x:a) x=x*ninv%NTTMOD; }
}

```

Same contract as FFT but exact (mod 998244353, which has a 2^{23} -th root of unity): pad to power of two, forward-transform both, multiply pointwise, inverse-transform. Output: exact convolution mod NTTMOD (no rounding error).

Generating Functions

```

// encode  $a_0, a_1, \dots$  as  $A(x) = \sum a_i x^i$ .
// sequence convolution  $(a*b)_n = \sum a_i b_{n-i} \leftrightarrow A(x)*B(x)$  -- use FFT/NTT.
//  $1/(1-x) = 1+x+x^2+\dots$ ;  $1/(1-x)^k = \sum C(n+k-1, k-1) x^n$  (stars and bars!)
// shift/scale/derivative of the sequence  $\leftrightarrow$  algebraic ops on  $A(x)$ .

```

No fixed I/O — model a combinatorial recurrence as a formal power series, manipulate algebraically, then read coefficients back off (often via FFT/NTT for the multiplication step).

B — Array Techniques

Difference Array

```

vector<ll> d(n+1,0);
void rangeAdd(ll l, ll r, ll v){ d[l]+=v; d[r+1]-=v; } // 0-indexed, [l,r]
void build(vector<ll>&a){ // after all rangeAdd calls
  a[0]=d[0];
  for(ll i=1;i<n;i++) a[i]=a[i-1]+d[i];
}

```

I/O: range-add updates (l, r, v) applied offline $\rightarrow$ final array after all updates, via prefix-summing the difference array in $O(n + q)$.

Sliding Window Technique

```

ll l=0; /* window state */
for(ll r=0;r<n;r++){
  // add a[r] to window state
  while(/* window invalid */ false){
    // remove a[l] from window state
    l++;
  }
  // window [l,r] is valid here; update answer
}

```

I/O: array $a[0..n-1]$ and a window-validity condition $\rightarrow$ best/count over all valid contiguous windows, in $O(n)$ amortized.

Two Pointers

```
ll i=0, j=n-1;
while(i<j){
    ll s=a[i]+a[j];
    if(s==target) { /* found */ i++; j--; }
    else if(s<target) i++;
    else j--;
}
```

I/O: sorted array a , target $\rightarrow$ pair (or count of pairs) summing to target, by moving pointers inward in $O(n)$.

Coordinate Compression

```
vector<ll> vals(a.begin(),a.end());
sort(vals.begin(),vals.end());
vals.erase(unique(vals.begin(),vals.end()),vals.end());
for(auto &x: a) x = lower_bound(vals.begin(),vals.end(),x)-vals.begin(); // 0-indexed
rank
```

I/O: array of up to 10^9 -range values $\rightarrow$ each value replaced by its rank among distinct values (0-indexed), so it can index a BIT/segment tree.

Direction Array

```
ll dx[]={1,-1,0,0}, dy[]={0,0,1,-1}; // 4-dir; add diagonals for 8-dir
for(ll d=0; d<4; d++){
    ll nx=x+dx[d], ny=y+dy[d];
    if(nx>=0&&nx<R&&ny>=0&&ny<C) { /* visit (nx,ny) */ }
}
```

I/O: grid of size $R \times C$, cell $(x, y) \rightarrow$ its (in-bounds) 4/8-connected neighbours, used inside BFS/DFS on a grid.

B — Graph Basics

Depth First Search (DFS)

```
vector<ll> g[MAXN]; bool vis[MAXN];
void dfs(ll u){
    vis[u]=1;
    for(ll v: g[u]) if(!vis[v]) dfs(v);
}
```

I/O: adjacency list g , start node $\rightarrow$ visits every reachable node once, $O(V + E)$. (Use iterative stack version if depth can exceed $\sim 10^5$.)

Breadth First Search (BFS)

```
vector<ll> dist(n,-1);
queue<ll> q; dist[s]=0; q.push(s);
while(!q.empty()){
    ll u=q.front(); q.pop();
    for(ll v: g[u]) if(dist[v]==-1){ dist[v]=dist[u]+1; q.push(v); }
}
```

I/O: adjacency list, source $s \rightarrow$ shortest unweighted distance to every node, $O(V+E)$.

Topological Sorting

```
vector<ll> indeg(n,0), order;
for(ll u=0;u<n;u++) for(ll v: g[u]) indeg[v]++;
queue<ll> q;
for(ll i=0;i<n;i++) if(!indeg[i]) q.push(i);
while(!q.empty()){
    ll u=q.front(); q.pop(); order.push_back(u);
    for(ll v: g[u]) if(--indeg[v]==0) q.push(v);
}
// if order.size() < n -> cycle exists
```

I/O: DAG adjacency list $\rightarrow$ a topological order (Kahn's algorithm); order shorter than n means the graph has a cycle.

Tree Diameter

```
pair<ll,ll> bfsFar(ll s, ll n){
    vector<ll> dist(n,-1); queue<ll> q; dist[s]=0; q.push(s);
    ll far=s;
    while(!q.empty()){ ll u=q.front(); q.pop();
        if(dist[u]>dist[far]) far=u;
        for(ll v: g[u]) if(dist[v]==-1){ dist[v]=dist[u]+1; q.push(v); }
    }
    return {far, dist[far]};
}
ll diameter(ll n){ auto [a,_]=bfsFar(0,n); auto [b,d]=bfsFar(a,n); return d; }
```

Input: tree adjacency list. Two BFS passes: from any node find the farthest node a , then from a find the farthest node b — $\text{dist}(a,b)$ is the diameter. Output: diameter length (and endpoints a,b).

Cycle Detection

```
ll state[MAXN]; // 0=unvisited, 1=in stack, 2=done -- directed graph
bool hasCycleDFS(ll u){
    state[u]=1;
    for(ll v: g[u]){
```

```

    if(state[v]==1) return true;
    if(state[v]==0 && hasCycleDFS(v)) return true;
}
state[u]=2;
return false;
}

```

// undirected: cycle iff DFS/BFS finds an edge to an already-visited non-parent node

I/O: directed adjacency list $\rightarrow$ true if a back-edge to a node currently on the DFS stack is found (3-colour DFS). For undirected graphs, check edges to a visited vertex that isn't the immediate parent.

DSU / Union Find

```

struct DSU{
    vector<ll> par, sz;
    DSU(ll n){ par.resize(n+1); sz.assign(n+1,1); for(ll i=0;i<=n;i++) par[i]=i; }
    ll find(ll x){ return par[x]==x? x: par[x]=find(par[x]); }
    bool merge(ll x, ll y){
        x=find(x); y=find(y);
        if(x==y) return false;
        if(sz[x]>sz[y]) swap(x,y);
        par[x]=y; sz[y]+=sz[x];
        return true;
    }
};

```

I/O: n (nodes), then $\text{merge}(x,y)$ / $\text{find}(x)$ calls $\rightarrow$ connectivity queries, $O(\alpha(n))$ amortized per op.

Minimum Spanning Tree (Prim's and Kruskal's)

```

// Kruskal: sort edges by weight, add if endpoints are in different DSU sets.
ll kruskal(ll n, vector<array<ll,3>>&edges /*{w,u,v}*/){
    sort(edges.begin(),edges.end());
    DSU dsu(n); ll cost=0;
    for(auto&[w,u,v]: edges) if(dsu.merge(u,v)) cost+=w;
    return cost;
}

// Prim's: grow tree from any node, add cheapest edge to a new node
// (priority_queue), repeat.
ll prim(ll n, vector<vector<pair<ll,ll>>>&g /*g[u]={v,w}*/, ll s=0){
    vector<bool> in(n,false); priority_queue<pair<ll,ll>,vector<pair<ll,ll>>,greater<>>
        pq;
    pq.push({0,s}); ll cost=0;
    while(!pq.empty()){
        auto [w,u]=pq.top(); pq.pop();
        if(in[u]) continue; in[u]=true; cost+=w;
        for(auto&[v,wt]: g[u]) if(!in[v]) pq.push({wt,v});
    }
    return cost;
}

```

I/O: weighted (connected) graph $\rightarrow$ total weight of a minimum spanning tree, $O(E \log E)$.

B — Searching & Sorting

Binary Search (incl. binary-search-on-answer)

```
ll lo=L, hi=R; // find smallest x in [L,R] with pred(x) true, pred monotonic false..
    true
while(lo<hi){
    ll mid=lo+(hi-lo)/2;
    if(pred(mid)) hi=mid; else lo=mid+1;
}
```

// lo == hi == answer (or R+1 / "not found" if pred never true -- check bounds)

I/O: search range $[L, R]$ and a monotonic predicate $\rightarrow$ smallest x with $\text{pred}(x)$ true, $O(\log(R - L))$ predicate calls.

Ternary Search / Binary Search on Answer

```
double lo=L, hi=R; // unimodal f on [L,R], find argmax
for(ll it=0; it<200; it++){
    double m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
    if(f(m1)<f(m2)) lo=m1; else hi=m2;
}
```

I/O: unimodal function f on $[L, R] \rightarrow$ (near-)optimal x maximizing/minimizing f , by shrinking the interval each iteration.

C++ STL Sort and Custom Comparator

```
sort(a.begin(), a.end(), [](const pair<ll,ll>&x, const pair<ll,ll>&y){
    if(x.first != y.first) return x.first < y.first;
    return x.second > y.second;
});
```

I/O: array of custom objects, comparator (must define a strict weak order) $\rightarrow$ sorted array, $O(n \log n)$.

B — Dynamic Programming

Knapsack and Basic Dynamic Programming

```
ll dp[MAXW+1]; // 0/1 knapsack, capacity W
void knapsack(vector<pair<ll,ll>>&items /*{wt,val}*/, ll W){
    for(auto&[w,v]: items)
        for(ll c=W; c>=w; c--) dp[c]=max(dp[c], dp[c-w]+v);
}
```

}

I/O: items (weight,value), capacity $W \rightarrow \max \text{ value}, O(nW)$. (Iterate c ascending instead for unbounded knapsack.)

Longest Increasing Subsequence (LIS)

```

11 lis(vector<ll>&a){
    vector<ll> t; // t[k] = smallest tail of an increasing subseq of length k+1
    for(ll x: a){
        auto it=lower_bound(t.begin(),t.end(),x); // upper_bound for non-decreasing
        if(it==t.end()) t.push_back(x); else *it=x;
    }
    return t.size();
}

```

I/O: array $a[0..n-1] \rightarrow \text{length of the longest strictly-increasing subsequence}, O(n \log n)$.

Interval DP / Range DP / Matrix Chain Multiplication

```

11 dp[MAXN][MAXN]; // dp[l][r] = best way to combine range [l,r]
void intervalDP(ll n){
    for(ll len=2; len<=n; len++){
        for(ll l=0; l+len-1<n; l++){
            ll r=l+len-1; dp[l][r]=INF;
            for(ll k=l; k<r; k++){
                dp[l][r]=min(dp[l][r], dp[l][k]+dp[k+1][r]+cost(l,k,r));
            }
        }
    }
}

```

I/O: n items with a pairwise merge cost function $\rightarrow$ optimal cost to combine the whole range, $O(n^3)$ (MCM: $\text{cost}(l,k,r) = p_l p_{k+1} p_{r+1}$).

Edit Distance

```

11 editDist(string&a, string&b){
    ll n=a.size(), m=b.size();
    vector<vector<ll>> dp(n+1, vector<ll>(m+1));
    for(ll i=0; i<=n; i++) dp[i][0]=i;
    for(ll j=0; j<=m; j++) dp[0][j]=j;
    for(ll i=1; i<=n; i++) for(ll j=1; j<=m; j++)
        dp[i][j] = a[i-1]==b[j-1]? dp[i-1][j-1]:
            1+min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    return dp[n][m];
}

```

I/O: strings $a, b \rightarrow \min \text{ insert/delete/replace ops to turn } a \text{ into } b, O(nm)$.

Bitmask DP

```

11 dp[1<<MAXK][MAXK]; // e.g. TSP: dp[mask][last] = min cost visiting set mask, ending
    at last
void bitmaskDP(11 k){
    memset(dp,0x3f,sizeof(dp));
    dp[1][0]=0;
    for(11 mask=1; mask<(1<<k); mask++){
        for(11 last=0; last<k; last++){
            if(!(mask&(1<<last)) || dp[mask][last]>=INF) continue;
            for(11 nxt=0; nxt<k; nxt++) if(!(mask&(1<<nxt)))
                dp[mask|(1<<nxt)][nxt]=min(dp[mask|(1<<nxt)][nxt], dp[mask][last]+cost(last,nxt));
        }
    }
}

```

I/O: $k \leq \sim 20$ items with pairwise costs $\rightarrow$ best full-order cost (e.g. TSP), $O(2^k k^2)$, state = (visited set, last item).

DP on Trees and DAGs

```

11 dp[MAXN]; bool vis[MAXN];
11 treeDP(11 u, 11 p){
    11 res = base(u);
    for(11 v: g[u]) if(v!=p) res = combine(res, treeDP(v,u));
    return dp[u]=res;
}

```

I/O: tree/DAG adjacency list, root $\rightarrow$ dp value per node via post-order DFS, combining children's answers, $O(V + E)$.

Sum Over Subsets DP (SOS DP)

```

11 f[1<<MAXK]; // f[mask] initially = value at exactly mask
void sos(11 k){
    for(11 i=0; i<k; i++){
        for(11 mask=0; mask<(1<<k); mask++){
            if(mask&(1<<i)) f[mask]+=f[mask^(1<<i)];
        }
        // now f[mask] = sum over all submasks of mask (of the original values)
    }
}

```

I/O: array $f[0..2^k - 1]$ indexed by bitmask $\rightarrow$ for every mask, sum of f over all its submasks, $O(k \cdot 2^k)$ instead of $O(3^k)$.

B — Data Structures

Segment Tree (Point Update Range Query)

```

struct SegTree{
    11 n; vector<11> st;
    SegTree(11 _n){ n=_n; st.assign(4*n+10,0); }
}

```

```

void update(ll node, ll l, ll r, ll pos, ll val){
    if(l==r){ st[node]=val; return; }
    ll m=(l+r)/2;
    if(pos<=m) update(2*node,l,m,pos,val); else update(2*node+1,m+1,r,pos,val);
    st[node]=st[2*node]+st[2*node+1]; // combine op (here: sum)
}
ll query(ll node, ll l, ll r, ll ql, ll qr){
    if(ql>r || qr<l) return 0; // identity of combine op
    if(ql<=l && qr>=r) return st[node];
    ll m=(l+r)/2;
    return query(2*node,l,m,ql,qr)+query(2*node+1,m+1,r,ql,qr);
}
void update(ll pos, ll val){ update(1,0,n-1,pos,val); }
ll query(ll l, ll r){ return query(1,0,n-1,l,r); }
};

```

I/O: array size n , then point-update(i, v) / range-query(l, r) calls $\rightarrow$ combined value (sum/min/max/...) over $[l, r]$, $O(\log n)$ per op.

Segment Tree with Lazy Propagation

```

struct LazySegTree{
    ll n; vector<ll> st, lz;
    LazySegTree(ll _n){ n=_n; st.assign(4*n+10,0); lz.assign(4*n+10,0); }
    void push(ll node, ll l, ll r){
        if(!lz[node]) return;
        st[node]+=(r-l+1)*lz[node];
        if(l!=r){ lz[2*node]+=lz[node]; lz[2*node+1]+=lz[node]; }
        lz[node]=0;
    }
    void update(ll node, ll l, ll r, ll ql, ll qr, ll val){
        push(node,l,r);
        if(ql>r || qr<l) return;
        if(ql<=l && qr>=r){ lz[node]+=val; push(node,l,r); return; }
        ll m=(l+r)/2;
        update(2*node,l,m,ql,qr,val); update(2*node+1,m+1,r,ql,qr,val);
        st[node]=st[2*node]+st[2*node+1];
    }
    ll query(ll node, ll l, ll r, ll ql, ll qr){
        push(node,l,r);
        if(ql>r || qr<l) return 0;
        if(ql<=l && qr>=r) return st[node];
        ll m=(l+r)/2;
        return query(2*node,l,m,ql,qr)+query(2*node+1,m+1,r,ql,qr);
    }
    void update(ll l, ll r, ll val){ update(1,0,n-1,l,r,val); }
    ll query(ll l, ll r){ return query(1,0,n-1,l,r); }
};

```

I/O: array size n , then range-add(l, r, v) / range-sum-query(l, r) calls $\rightarrow$ updated sum, $O(\log n)$ per op via deferred (lazy) pushes.

BIT / Fenwick Tree

```

struct BIT{

```

```

ll n; vector<ll> t;
BIT(ll _n){ n=_n; t.assign(n+1,0); }
void update(ll i, ll v){ for(++i; i<=n; i+=i&(-i)) t[i]+=v; } // 0-indexed pos
ll query(ll i){ ll s=0; for(++i; i>0; i-=i&(-i)) s+=t[i]; return s; } // sum [0..i]
ll range(ll l, ll r){ return query(r)-(l? query(l-1):0); }
};

```

I/O: array size n , then point-update(i, v) / prefix or range-sum queries $\rightarrow$ sum, $O(\log n)$ per op, $\sim 4\times$ smaller constant than a segment tree.

Lower bound on BIT (BIT binary search)

```

ll lowerBoundBIT(BIT&bit, ll target){
    // smallest prefix index with prefix-sum >= target (all vals>=0)
    ll pos=0, LOG=__lg(bit.n)+1;
    for(ll j=LOG; j>=0; j--){
        ll nxt=pos+(1<<j);
        if(nxt<=bit.n && bit.t[nxt]<target){ pos=nxt; target-=bit.t[nxt]; }
    }
    return pos; // 0-indexed: first position where cumulative sum reaches target
}

```

Input: a BIT over non-negative values and a target prefix sum. Walks the BIT's internal binary-lifting structure instead of binary-searching with log queries. Output: smallest index whose prefix sum $\geq$ target, $O(\log n)$.

Sparse Table

```

ll sp[LOG][MAXN];
void buildSparse(vector<ll>&a){
    ll n=a.size();
    for(ll i=0; i<n; i++) sp[0][i]=a[i];
    for(ll j=1; (1<<j)<=n; j++){
        for(ll i=0; i+(1<<j)<=n; i++){
            sp[j][i]=min(sp[j-1][i], sp[j-1][i+(1<<(j-1))]);
        }
    }
    ll queryMin(ll l, ll r){ // [l,r], idempotent op only (min/max/gcd/and/or)
        ll j=__lg(r-l+1);
        return min(sp[j][l], sp[j][r-(1<<j)+1]);
    }
}

```

I/O: static array a , then range-min (or max/gcd/and/or) queries $\rightarrow$ answer in $O(1)$ after $O(n \log n)$ build (no updates allowed).

Basic SQRT Decomposition

```

ll BLK; vector<ll> a, blockSum;
void build(){ BLK=max(1.0, sqrt(a.size())); blockSum.assign(a.size()/BLK+1,0);
    for(ll i=0; i<(ll)a.size(); i++) blockSum[i/BLK]+=a[i]; }
void update(ll i, ll v){ blockSum[i/BLK]+=v-a[i]; a[i]=v; }
ll query(ll l, ll r){ // sum [l,r]
    ll s=0;
    while(l<=r){

```

```

    if(l%BLK==0 && l+BLK-1<=r){ s+=blockSum[l/BLK]; l+=BLK; }
    else { s+=a[l]; l++; }
}
return s;
}

```

I/O: array, then point-update(i, v) / range-query(l, r) calls $\rightarrow$ answer in $O(\sqrt{n})$ per op; use when the query can't be expressed with a plain segment tree.

Splitting Objects into Light and Heavy / Case Processing

*// classify by size/degree/frequency threshold T (often $T=\sqrt{n}$ or $\sqrt{q}$):
 // "heavy" objects (count $\leq n/T$) get a precomputed/expensive-but-rare treatment,
 // "light" objects (count can be large, but each op is cheap) handled directly.
 // total work bounded by ($\#heavy * expensive$) + ($\#ops * cheap$).*

No fixed I/O — a general $\sqrt{q}$ -decomposition style trick: pick a threshold so the two cases' total costs balance, e.g. heavy-light decomposition on tree paths, or heavy vs light vertices in DSU-on-tree / small-to-large merging.

Trie on Strings and Bits

```

struct Trie{
    vector<array<ll,2>> ch; // binary trie (bits); use array<ll,26> for strings
    vector<ll> cnt;
    Trie(){ ch.push_back({-1,-1}); cnt.push_back(0); }
    void insert(ll x, ll BITS=30){
        ll u=0;
        for(ll b=BITS; b>=0; b--){
            ll bit=(x>>b)&1;
            if(ch[u][bit]==-1){ ch[u][bit]=ch.size(); ch.push_back({-1,-1}); cnt.push_back(0); }
            u=ch[u][bit]; cnt[u]++;
        }
    }
    ll maxXor(ll x, ll BITS=30){
        ll u=0, res=0;
        for(ll b=BITS; b>=0; b--){
            ll bit=(x>>b)&1, want=bit^1;
            if(ch[u][want]!=-1){ res|=(1LL<<b); u=ch[u][want]; } else u=ch[u][bit];
        }
        return res;
    }
};

```

I/O: a set of strings/integers to insert, then queries (prefix count / max-XOR with x) $\rightarrow$ answer, $O(\text{length})$ per op.

B — Strings

String Hashing

```
const ll B1=131, M1=1e9+7;
vector<ll> pw1, h1;
void buildHash(string&s){
    ll n=s.size(); pw1.assign(n+1,1); h1.assign(n+1,0);
    for(ll i=1;i<=n;i++) pw1[i]=pw1[i-1]*B1%M1;
    for(ll i=0;i<n;i++) h1[i+1]=(h1[i]*B1+s[i])%M1;
}
ll getHash(ll l, ll r){ // [l,r] 0-indexed
    return ((h1[r+1]-h1[l]*pw1[r-l+1])%M1+M1)%M1;
}
```

I/O: string s , then substring-hash(l, r) queries $\rightarrow$ hash value, $O(1)$ per query after $O(n)$ build; compare two substrings in $O(1)$ (use 2 hashes/mods to cut collision risk for adversarial data).

Knuth Morris Pratt (KMP) and Prefix Automaton

```
vector<ll> prefixFunc(string&s){ // pi[i] = longest proper prefix==suffix of s[0..i]
    ll n=s.size(); vector<ll> pi(n,0);
    for(ll i=1;i<n;i++){
        ll j=pi[i-1];
        while(j>0 && s[i]!=s[j]) j=pi[j-1];
        if(s[i]==s[j]) j++;
        pi[i]=j;
    }
    return pi;
}
vector<ll> kmpSearch(string&text, string&pat){ // returns match start positions
    string s=pat+"#"+text; vector<ll> pi=prefixFunc(s), res;
    for(ll i=pat.size()+1;i<(ll)s.size();i++)
        if(pi[i]==(ll)pat.size()) res.push_back(i-2*pat.size());
    return res;
}
```

I/O: text, pattern $\rightarrow$ all starting indices of pattern in text, $O(|text| + |pattern|)$, via the prefix function (failure links).

Z algorithm

```
vector<ll> zFunc(string&s){ // z[i] = longest common prefix of s and s[i..]
    ll n=s.size(); vector<ll> z(n,0); z[0]=n;
    ll l=0,r=0;
    for(ll i=1;i<n;i++){
        if(i<r) z[i]=min(r-i, z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]>r){ l=i; r=i+z[i]; }
    }
    return z;
}
```

I/O: string $s \rightarrow z[i]$ = length of the longest common prefix of s and $s[i..]$, $O(n)$; find pattern via z on $pattern+\# + text$.

Aho Corasick

```
struct AhoCorasick{
    vector<array<ll,26>> ch; vector<ll> fail, out; // out[u] = #patterns ending at/
        through u
    AhoCorasick(){ ch.push_back({}); ch[0].fill(0); fail.push_back(0); out.push_back(0);
    }
    void insert(string&s){
        ll u=0;
        for(char c: s){
            ll x=c-'a';
            if(!ch[u][x]){
                ch[u][x]=ch.size();
                ch.push_back({}); ch.back().fill(0);
                fail.push_back(0); out.push_back(0);
            }
            u=ch[u][x];
        }
        out[u]++;
    }
    void build(){ // BFS to set fail links and compress transitions
        queue<ll> q;
        for(ll c=0;c<26;c++) if(ch[0][c]) q.push(ch[0][c]);
        while(!q.empty()){
            ll u=q.front(); q.pop(); out[u]+=out[fail[u]];
            for(ll c=0;c<26;c++){
                ll v=ch[u][c];
                if(!v){ ch[u][c]=ch[fail[u]][c]; continue; }
                fail[v]=ch[fail[u]][c]; q.push(v);
            }
        }
    }
};
```

Input: a dictionary of patterns (insert each, then build()), and a text streamed char by char through ch[[]]. Builds a trie plus failure links (like KMP, generalized to multiple patterns) so every text position advances in $O(1)$ amortized. Output: $out[current\ node] = \# \text{ dictionary patterns matching ending here.}$

B — Flows & Matching

Maximum Flow / Minimum Cut (Dinic's)

```
struct Dinic{
    struct E{ ll to, cap, flow; };
    vector<E> edges; vector<vector<ll>> g; vector<ll> lvl, it; ll n;
    Dinic(ll _n):n(_n){ g.resize(n); }
    void addEdge(ll u, ll v, ll c){
        g[u].push_back(edges.size()); edges.push_back({v,c,0});
        g[v].push_back(edges.size()); edges.push_back({u,0,0});
    }
};
```

```

bool bfs(ll s, ll t){
    lvl.assign(n,-1); queue<ll> q; lvl[s]=0; q.push(s);
    while(!q.empty()){ ll u=q.front(); q.pop();
        for(ll id: g[u]){ auto&e=edges[id];
            if(lvl[e.to]<0 && e.cap>e.flow){ lvl[e.to]=lvl[u]+1; q.push(e.to); } } }
    return lvl[t]>=0;
}
ll dfs(ll u, ll t, ll pushed){
    if(u==t || !pushed) return pushed;
    for(ll&i=it[u]; i<(ll)g[u].size(); i++){
        ll id=g[u][i]; auto&e=edges[id];
        if(lvl[e.to]!=lvl[u]+1 || e.cap<=e.flow) continue;
        ll d=dfs(e.to, t, min(pushed, e.cap-e.flow));
        if(d>0){ e.flow+=d; edges[id^1].flow-=d; return d; }
    }
    return 0;
}
ll maxflow(ll s, ll t){
    ll flow=0;
    while(bfs(s,t)){
        it.assign(n,0);
        while(ll pushed=dfs(s,t,LLONG_MAX)) flow+=pushed;
    }
    return flow;
}
};

```

I/O: directed graph with edge capacities, source s , sink $t \rightarrow$ max flow value (= min cut by max-flow min-cut theorem), $O(V^2E)$.

Bipartite Matching (Hopcroft-Karp and Kuhn's)

```

// Kuhn's: augmenting path per left vertex,  $O(V*E)$ 
vector<ll> matchR; vector<bool> used;
bool tryKuhn(ll u, vector<vector<ll>>&g){
    for(ll v: g[u]){
        if(used[v]) continue; used[v]=true;
        if(matchR[v]==-1 || tryKuhn(matchR[v], g)){ matchR[v]=u; return true; }
    }
    return false;
}
ll kuhnMatching(ll nLeft, ll nRight, vector<vector<ll>>&g){
    matchR.assign(nRight,-1); ll res=0;
    for(ll u=0;u<nLeft;u++){ used.assign(nRight,false); if(tryKuhn(u,g)) res++; }
    return res;
}

```

I/O: bipartite adjacency list (left $\rightarrow$ right) $\rightarrow$ max matching size, $O(VE)$ (use Hopcroft-Karp, $O(E\sqrt{V})$, if V is large).

B — Advanced Graph

Strongly Connected Components (SCC, Kosaraju)

```
vector<ll> g[MAXN], rg[MAXN], order_, comp; bool vis[MAXN];
void dfs1(ll u){ vis[u]=1; for(ll v: g[u]) if(!vis[v]) dfs1(v); order_.push_back(u); }
void dfs2(ll u, ll c){ comp[u]=c; for(ll v: rg[u]) if(comp[v]==-1) dfs2(v,c); }
ll kosaraju(ll n){
    fill(vis,vis+n,0);
    for(ll i=0;i<n;i++) if(!vis[i]) dfs1(i);
    comp.assign(n,-1); ll c=0;
    for(ll i=n-1;i>=0;i--){ ll u=order_[i]; if(comp[u]==-1) dfs2(u,c++); }
    return c; // #SCCs; comp[u] gives component id (reverse-topological order)
}
```

I/O: directed graph $\rightarrow$ SCC id per node and #SCCs, $O(V+E)$ via two DFS passes (finish-order, then on the reverse graph).

2-SAT

```
// literal x -> node 2x, !x -> node 2x+1; clause (a OR b) => edges !a->b, !b->a
ll n2; vector<ll> g2[2*MAXN], rg2[2*MAXN], comp2;
void addClause(ll a, bool av, ll b, bool bv){
    ll na=2*a+(av?0:1), nna=2*a+(av?1:0);
    ll nb=2*b+(bv?0:1), nnb=2*b+(bv?1:0);
    g2[nna].push_back(nb); rg2[nb].push_back(nna);
    g2[nnb].push_back(na); rg2[na].push_back(nnb);
}
bool solve2SAT(ll n){ // run kosaraju on g2/rg2 over 2n nodes, fill comp2
    // ...
    for(ll i=0;i<n;i++) if(comp2[2*i]==comp2[2*i+1]) return false; // UNSAT
    return true; // assignment: x_i = true iff comp2[2*i] > comp2[2*i+1] (later in topo order)
}
```

Input: boolean variables and 2-literal clauses (via addClause). Builds the implication graph ($\neg a \Rightarrow b$ etc.) and runs SCC. Output: satisfiable iff no variable's two literals share an SCC; the assignment follows the SCCs' reverse-topological order.

Articulation Bridges and Bridge Tree

```
ll disc[MAXN], low[MAXN], timer_=0; vector<pair<ll,ll>> bridges;
void dfsBridge(ll u, ll p){
    disc[u]=low[u]=timer_++;
    for(ll v: g[u]){
        if(v==p) continue;
        if(disc[v]!=-1) low[u]=min(low[u],disc[v]);
        else { dfsBridge(v,u); low[u]=min(low[u],low[v]);
            if(low[v]>disc[u]) bridges.push_back({u,v}); }
    }
}
```

I/O: undirected graph $\rightarrow$ list of bridge edges ($low[v] > disc[u]$ means edge (u,v) isn't on any cycle), $O(V+E)$; contracting each 2-edge-connected component to a node gives the bridge tree.

Articulation Points and Block Cut Tree

```

11 disc2[MAXN], low2[MAXN], timer2=0; vector<bool> isAP;
void dfsAP(11 u, 11 p, 11 root){
    disc2[u]=low2[u]=timer2++; 11 children=0;
    for(11 v: g[u]){
        if(v==p) continue;
        if(disc2[v]!=-1){ low2[u]=min(low2[u],disc2[v]); continue; }
        children++; dfsAP(v,u,root); low2[u]=min(low2[u],low2[v]);
        if(u!=root && low2[v]>=disc2[u]) isAP[u]=true;
    }
    if(u==root && children>1) isAP[u]=true;
}

```

I/O: undirected graph $\rightarrow$ which nodes are cut vertices (removing them disconnects the graph), $O(V + E)$; grouping nodes by biconnected component (between consecutive articulation points) gives the block-cut tree.

Heavy Light Decomposition (HLD) — Subtrees and Path Query

```

11 par[MAXN], dep[MAXN], heavy[MAXN], headOf[MAXN], posInSeg[MAXN], subSz[MAXN],
    curPos=0;
11 dfsSz(11 u, 11 p){
    par[u]=p; subSz[u]=1; 11 best=0;
    for(11 v: g[u]) if(v!=p){
        dep[v]=dep[u]+1; 11 sz=dfsSz(v,u); subSz[u]+=sz;
        if(sz>best){ best=sz; heavy[u]=v; }
    }
    return subSz[u];
}
void decompose(11 u, 11 h){
    headOf[u]=h; posInSeg[u]=curPos++;
    if(heavy[u]!=-1) decompose(heavy[u], h);
    for(11 v: g[u]) if(v!=par[u] && v!=heavy[u]) decompose(v, v);
}
// path query(u,v): while headOf[u]!=headOf[v], jump the deeper head up,
// query segtree on posInSeg range, then move to par[head]; finally
// query posInSeg[min(dep)..posInSeg[max(dep)]]

```

Input: tree adjacency list, root. Decomposes into heavy chains so any root-to-node (or u-to-v) path touches $O(\log n)$ contiguous segment-tree ranges. Output: posInSeg[] / headOf[] used to answer path or subtree queries in $O(\log^2 n)$ per query (subtree = one contiguous range).

Centroid Decomposition

```

11 subSz2[MAXN]; bool removed[MAXN];
11 calcSz(11 u, 11 p){
    subSz2[u]=1;
    for(11 v: g[u]) if(v!=p && !removed[v]) subSz2[u]+=calcSz(v,u);
    return subSz2[u];
}
11 findCentroid(11 u, 11 p, 11 treeSz){

```

```

    for(ll v: g[u]) if(v!=p && !removed[v] && subsz2[v]>treeSz/2) return findCentroid(v,
        u,treeSz);
    return u;
}
void decompose2(ll u){
    ll treeSz=calcSz(u,-1);
    ll c=findCentroid(u,-1,treeSz);
    removed[c]=true;
    // process all paths through c here (e.g. count paths of given length via two-
        pointer/map)
    for(ll v: g[c]) if(!removed[v]) decompose2(v);
}

```

Input: tree adjacency list. Repeatedly finds each remaining component's centroid (vertex whose removal leaves subtrees $\leq$ half size) and recurses on the pieces, $O(n \log n)$ total. Output: a centroid decomposition tree (depth $O(\log n)$), typically used to answer "count/find paths satisfying X " queries.

Binary Lifting and LCA

```

ll up[LOG][MAXN], depth_[MAXN];
void dfsLCA(ll u, ll p, ll d){
    depth_[u]=d; up[0][u]=p;
    for(ll j=1;j<LOG;j++) up[j][u] = up[j-1][u]==-1? -1: up[j-1][up[j-1][u]];
    for(ll v: g[u]) if(v!=p) dfsLCA(v,u,d+1);
}
ll lca(ll u, ll v){
    if(depth_[u]<depth_[v]) swap(u,v);
    ll diff=depth_[u]-depth_[v];
    for(ll j=0;j<LOG;j++) if(diff&(1<<j)) u=up[j][u];
    if(u==v) return u;
    for(ll j=LOG-1;j>=0;j--) if(up[j][u]!=up[j][v]){ u=up[j][u]; v=up[j][v]; }
    return up[0][u];
}

```

I/O: tree adjacency list, root, then $\text{lca}(u,v)$ queries $\rightarrow$ lowest common ancestor, $O(\log n)$ per query after $O(n \log n)$ build; $\text{dist}(u,v) = \text{depth}[u] + \text{depth}[v] - 2 \cdot \text{depth}[\text{lca}]$.

LCA in $O(1)$ (optional stretch, sparse table on Euler tour)

*// Euler tour: push node on entry and after each child's return $\Rightarrow 2n-1$ entries.
 // range-min (by depth) over the tour between first-occurrences of u,v = their LCA.
 // build a sparse table (see Sparse Table) over (depth, node) pairs of the tour.*

Input: Euler tour of the tree with depths, first-occurrence index of each node. Output: $\text{LCA}(u,v)$ in $O(1)$ via a range-min sparse table over the tour, after $O(n \log n)$ preprocessing (vs. $O(\log n)$ for plain binary lifting).

B — DP Optimization

Convex Hull Trick (CHT) and Dynamic CHT

```
struct Line{ ll m,b; double isectX(Line o){ return (double)(o.b-b)/(m-o.m); } };
struct CHT{ // monotonic CHT: slopes added in decreasing order, queries increasing x
    vector<Line> lines;
    void add(ll m, ll b){
        while(lines.size()>=2 && lines.back().isectX(lines[lines.size()-2]) <=
            Line{m,b}.isectX(lines[lines.size()-2])) lines.pop_back();
        lines.push_back({m,b});
    }
    ll ptr=0;
    ll query(ll x){ // call with x non-decreasing
        if(ptr>=(ll)lines.size()) ptr=lines.size()-1;
        while(ptr+1<(ll)lines.size() &&
            lines[ptr+1].m*x+lines[ptr+1].b >= lines[ptr].m*x+lines[ptr].b) ptr++;
        return lines[ptr].m*x+lines[ptr].b;
    }
};
```

// dynamic CHT (arbitrary insert/query order): use a std::set/Li-Chao tree instead.

I/O: lines $y = mx + b$ added (monotonic slopes), then query(x) (monotonic x) $\rightarrow$ max $mx + b$ over all added lines, amortized $O(1)$ per op; use a Li-Chao tree for arbitrary order, $O(\log(\text{range}))$ per op.

Knuth Optimization

```
// speeds up interval DP dp[l][r]=min_k dp[l][k]+dp[k+1][r]+C(l,r)
// to O(n^2) IF C satisfies the quadrangle inequality, by proving the optimal
// split point opt[l][r] is monotonic: opt[l][r-1] <= opt[l][r] <= opt[l+1][r].
ll opt[MAXN][MAXN];
for(ll len=2; len<=n; len++){
    for(ll l=0; l+len-1<n; l++){
        ll r=l+len-1; dp[l][r]=INF;
        for(ll k=opt[l][r-1]; k<=min(r-1,opt[l+1][r]); k++){
            if(dp[l][k]+dp[k+1][r]+C(l,r) < dp[l][r]){
                dp[l][r]=dp[l][k]+dp[k+1][r]+C(l,r); opt[l][r]=k;
            }
        }
    }
}
```

Input: same setup as Interval DP, with cost C satisfying the quadrangle inequality. Narrows each $dp[l][r]$'s split search to $[opt[l][r-1], opt[l+1][r]]$ instead of all of $[l, r]$. Output: same DP values, in $O(n^2)$ instead of $O(n^3)$.

Divide and Conquer Optimization

```
// speeds up dp[i][j] = min_{k<j} dp[i-1][k] + C(k,j) (layer i from layer i-1)
// to O(n log n) per layer IF opt(i,j) (argmin k) is monotonic in j.
void solve(ll i, ll lo, ll hi, ll optLo, ll optHi){
    if(lo>hi) return;
    ll mid=(lo+hi)/2, bestK=optLo;
    for(ll k=optLo; k<=min(mid,optHi); k++){
        if(dp[i-1][k]+C(k,mid) < dp[i][mid]){ dp[i][mid]=dp[i-1][k]+C(k,mid); bestK=k; }
    }
    solve(i,lo,mid-1,optLo,bestK); solve(i,mid+1,hi,bestK,optHi);
}
```

}

Input: per-layer DP transition $dp[i][j] = \min_{k < j} dp[i-1][k] + C(k, j)$ with monotonic optimal k . Recurses on j , narrowing the k -search range each half using the midpoint's optimal k as a bound. Output: layer i of the DP, $O(n \log n)$ instead of $O(n^2)$ per layer.
